Programación de Sistemas en Tiempo Real

Guía de estudio completa

Teoría desde cero, prácticas, exámenes resueltos, demostraciones, autoevaluación y simulacros

Hojas de problemas resueltas paso a paso.

Los simuladores interactivos están en la versión web de la guía; aquí aparecen como capturas.

Última actualización: 2026-11-08

Índice general

1 Hojas de problemas resueltas **2**

1.1 Hoja 1 · Cálculo numérico y bucles 2

1.2 Hoja 2 · Conjuntos y cadenas 3

1.3 Hoja 3 · Clases 4

1 Hojas de problemas resueltas

Ejercicios de las hojas de problemas, con el mismo formato que los exámenes: enunciado, qué piden, pasos plegables y resultado final. El código completo acompaña a la guía como ficheros independientes. Todos los resultados se han comprobado compilando y ejecutando.

1.1 Hoja 1 · Cálculo numérico y bucles

Leibniz, sumas parciales y criba de Eratóstenes

Enunciado

1. Calcula $\pi/4$ con la serie de Leibniz, $\sum_{n\geq 0}(-1)^n/(2n+1)$, con un millón de términos. Compara con `M_PI_4` usando `==` y usando `fabs`.
2. Dado un `vector<int>`, construye otro cuya posición `i` contenga la suma de los elementos desde `i` hasta el final.
3. Implementa la criba de Eratóstenes hasta 1 000 000 con un `vector<char>` que marque cada número como desconocido, primo o compuesto. Cuenta los primos y muestra los últimos ocho.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Tres bucles clásicos. Lo importante: entender **por qué == falla con reales**, recorrer un vector hacia atrás con un acumulador y hacer la criba en tiempo casi lineal.
Teoría que usas. Coma flotante (T7), `vector`, `iota`, bucles con `size_t` (sin signo: cuidado con `i >= 0`).

1 Serie de Leibniz y comparación

Se alterna el signo con `n % 2`. Con N términos, el error de la serie alternada es del orden de $1/(4N)$: con $N = 10^6$, unos $2{,}5 \cdot 10^{-7}$. Por eso `==` da `false` (el resultado **no puede** coincidir bit a bit con `M_PI_4`) pero `fabs(a-b) < 1e-6` da `true`.

CPP

```cpp
// 1) Leibniz: pi/4 = sum_{n>=0} (-1)^n / (2n+1)
double pi_4 = 0;
for (int n = 0; n < 1000000; ++n) pi_4 += (n % 2 == 0 ? 1.0 : -1.0) / (2 * n + 1);
cout << boolalpha << "pi_4 == M_PI_4 ? " << (pi_4 == M_PI_4) << '\n';
cout << setprecision(12) << pi_4 << " vs " << M_PI_4 << '\n';
cout << "|diferencia| < 1e-6 ? " << (fabs(pi_4 - M_PI_4) < 1e-6) << '\n';
```

2 Sumas desde `i` hasta el final

Se recorre **de atrás hacia delante** con un acumulador: `sumas[i] = sumas[i+1] + v[i]`. Coste $O(n)$ (frente a $O(n^2)$ si sumas cada vez desde `i`). Se usa `int i` y no `size_t i` porque el bucle termina con `i >= 0`: con un tipo sin signo, `i >= 0` es siempre cierto y el bucle no acaba.

CPP

```cpp
// 2) sumas parciales hasta el final (suma desde i hasta el último)
vector<int> v(10);
iota(v.begin(), v.end(), 1);
vector<int> sumas(v.size());
int acum = 0;
for (int i = static_cast<int>(v.size()) - 1; i >= 0; --i) { acum += v[i]; sumas[i] = acum; }
for (int x : sumas) cout << x << ' ';
cout << '\n';
```

3 Criba de Eratóstenes

Marcas iniciales: `'d'` (desconocido). Se recorre desde 2: si `i` sigue desconocido es primo y se marcan como compuestos sus múltiplos. **Se empieza en `i*i`**: los múltiplos menores, $k \cdot i$ con $k < i$, ya los marcó un primo más pequeño. Coste $O(n \log \log n)$.

CPP

```
    // 3) criba de Eratóstenes ('d' desconocido, 'p' primo, 'c' compuesto)
    const size_t N = 1000000;
    vector<char> marca(N + 1, 'd');
    marca[0] = marca[1] = 'c';
    for (size_t i = 2; i <= N; ++i) {
        if (marca[i] != 'd') continue;
        marca[i] = 'p';
        for (size_t m = i * i; m <= N; m += i) marca[m] = 'c';     // desde i*i: los anteriores ya están marcados
    }
    long cuenta = 0;
    for (char c : marca) cuenta += (c == 'p');
    cout << "primos hasta 1000000: " << cuenta << '\n';
    cout << "ultimos 8:";
    int mostrados = 0;
    vector<size_t> ult;
    for (size_t i = N; i > 1 && mostrados < 8; --i) if (marca[i] == 'p') { ult.push_back(i); ++mostrados; }
    for (auto it = ult.rbegin(); it != ult.rend(); ++it) cout << ' ' << *it;
    cout << '\n';
}
```

4 Comprobación

- Hay 78 498 primos por debajo de 1 000 000 (valor conocido) y el mayor es 999 983: coincide con la salida.
- Sumas parciales de $1..10$: $55, 54, 52, \ldots, 10$ (suma total 55, último elemento 10).

Resultado final

Salida

```
pi_4 == M_PI_4 ? false
0.785397913397 vs 0.785398163397
|diferencia| < 1e-6 ? true
55 54 52 49 45 40 34 27 19 10
primos hasta 1000000: 78498
ultimos 8: 999907 999917 999931 999953 999959 999961 999979 999983
```

Interpretación. `pi_4 == M_PI_4` es falso aunque la aproximación sea buena: la serie solo converge (lentamente) y además cada suma redondea. Compara siempre con tolerancia.

1.2 Hoja 2 · Conjuntos y cadenas

Operaciones con `set<int>` y caracteres repetidos

Enunciado

1. Escribe funciones que reciban dos `set<int>` y devuelvan su unión, su intersección y su diferencia, y otra que diga si el primero es subconjunto del segundo.
2. Escribe una función que diga si una cadena tiene caracteres repetidos, de tres maneras: con un `set`, con un `vector`/`map` y sin ninguna colección.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Usar `set` (ordenado, sin duplicados) y conocer sus operaciones: `insert`, `count`, rangos. Saber que `insert` **devuelve un `pair`** cuyo `.second` indica si se insertó.
Teoría que usas. `set<T>`, iteradores, `std::includes` (requiere rangos **ordenados**: un `set` lo está), `operator<<` sobrecargado.

1 Unión, intersección y diferencia

- Unión: copia de `a` y se insertan los de `b`. El `set` descarta duplicados.
- Intersección: de `a`, los que `b.count(x)` confirma.
- Diferencia: de `a`, los que **no** están en `b`.

Se pasa por `const&` para no copiar. Coste $O(n \log m)$.

CPP

```
set<int> unir(const set<int>& a, const set<int>& b) { set<int> r{a}; r.insert(b.begin(), b.end()); return r; }
set<int> interseccion(const set<int>& a, const set<int>& b) {
```

```
    set<int> r; for (int x : a) if (b.count(x)) r.insert(x); return r;
}
set<int> diferencia(const set<int>& a, const set<int>& b) {
    set<int> r; for (int x : a) if (!b.count(x)) r.insert(x); return r;
}
bool es_subconjunto(const set<int>& a, const set<int>& b) {     // ¿a está contenido en b?
    return includes(b.begin(), b.end(), a.begin(), a.end());
```

2 Subconjunto

`includes(b.begin(), b.end(), a.begin(), a.end())` devuelve `true` si todos los elementos de a están en `b`. Funciona porque ambos rangos están ordenados.

CPP

```
bool es_subconjunto(const set<int>& a, const set<int>& b) {     // ¿a está contenido en b?
    return includes(b.begin(), b.end(), a.begin(), a.end());
}
```

3 Caracteres repetidos

- **Con set**: `vistos.insert(c).second == false` significa que ya estaba: repetido. $O(n \log n)$.
- **Sin colecciones**: dos bucles anidados, $O(n^2)$, sin memoria extra.

(Con `map<char,int>` o `vector<int>(256)` se cuenta cada carácter y se busca un contador ≥ 2: $O(n)$.)

CPP

```
bool repetidos_set(const string& s) {              // con set: insert devuelve .second == false si ya estaba
    set<char> vistos;
    for (char c : s) if (!vistos.insert(c).second) return true;
    return false;
}
bool repetidos_sin_colecciones(const string& s) {  // sin colecciones: dos bucles anidados
    for (size_t i = 0; i < s.size(); ++i)
        for (size_t j = i + 1; j < s.size(); ++j) if (s[i] == s[j]) return true;
    return false;
}
```

4 Prueba

CPP

```
int main() {
    set<int> a{1, 2, 3, 4}, b{3, 4, 5};
    cout << "a=" << a << " b=" << b << '\n';
    cout << "union: " << unir(a, b) << "  interseccion: " << interseccion(a, b) << "  a-b: " << diferencia(a, b) <<
    ↪  '\n';
    cout << boolalpha << "{3,4} en a: " << es_subconjunto({3, 4}, a) << ", b en a: " << es_subconjunto(b, a) << '\n';
    for (string s : {"", "hola", "hello"})
        cout << "\"" << s << "\": set=" << repetidos_set(s) << " sin colecciones=" << repetidos_sin_colecciones(s) <<
        ↪  '\n';
}
```

Resultado final

Salida

```
a={1, 2, 3, 4} b={3, 4, 5}
union: {1, 2, 3, 4, 5}  interseccion: {3, 4}  a-b: {1, 2}
{3,4} en a: true, b en a: false
"": set=false sin colecciones=false
"hola": set=false sin colecciones=false
"hello": set=true sin colecciones=true
```

1.3 Hoja 3 · Clases

Esfera, punto y caja

Enunciado

1. Diseña la clase `Sphere` (diámetro y centro como `Point_3D`) con área, volumen, `contiene(punto)` y `en_superficie(punto)`, y `operator<<`.
2. Diseña la clase `Box` con alto, ancho y profundidad, que permita insertar objetos por volumen mientras quepan, saber si está llena y vaciarla.
3. Lee un valor positivo de `cin` sin que el programa se rompa si el usuario escribe letras.

Antes de empezar: qué piden y qué teoría usas

Qué piden. Clases con **invariantes** (el volumen libre nunca es negativo), métodos `const`, `friend operator<<` y entrada robusta.
Teoría que usas. Constructores con valores por defecto, lista de inicialización, `const` correcto, comparar reales con tolerancia, estado de error de `cin` (`clear` + `ignore`).

1 `Point_3D` y `Sphere`

La distancia entre dos puntos es $\sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$. Un punto está **dentro** si la distancia al centro es menor que el radio, y **en la superficie** si difiere del radio en menos de una tolerancia (nunca == con reales). Área $4\pi r^2$ y volumen $\frac{4}{3}\pi r^3$, con $r = d/2$.

CPP

```
class Point_3D {
public:
    Point_3D(double x = 0, double y = 0, double z = 0) : x_{x}, y_{y}, z_{z} {}
    double distancia(const Point_3D& o) const { return sqrt(pow(x_ - o.x_, 2) + pow(y_ - o.y_, 2) + pow(z_ - o.z_,
    ↪  2)); }
    friend ostream& operator<<(ostream& os, const Point_3D& p) { return os << '(' << p.x_ << ", " << p.y_ << ", " <<
    ↪  p.z_ << ')'; }
private:
    double x_, y_, z_;
};

class Sphere {
public:
    explicit Sphere(double diametro, Point_3D centro = {}) : d_{diametro}, c_{centro} {}
    double diametro() const { return d_; }
    void set_diametro(double d) { d_ = d; }
    double area() const { double r = d_ / 2; return 4 * M_PI * r * r; }
    double volumen() const { double r = d_ / 2; return 4.0 / 3 * M_PI * r * r * r; }
    // dentro: distancia < r; sobre la superficie: |distancia - r| < eps (nunca ==, son reales)
    bool contiene(const Point_3D& p) const { return c_.distancia(p) < d_ / 2; }
    bool en_superficie(const Point_3D& p, double eps = 1e-9) const { return fabs(c_.distancia(p) - d_ / 2) < eps; }
    friend ostream& operator<<(ostream& os, const Sphere& s) {
        return os << "Esfera d=" << s.d_ << " centro " << s.c_ << " area=" << s.area() << " volumen=" << s.volumen();
    }
private:
    double d_;
    Point_3D c_;
};
```

2 `Box`

El volumen libre empieza en $a \cdot w \cdot p$. `insertar_objeto` rechaza volúmenes ≤ 0 o mayores que el libre (devuelve `false` y **no cambia el estado**). `llena_` se actualiza al llegar a 0.

CPP

```
class Box {
public:
    Box(double alto, double ancho, double prof) : a_{alto}, w_{ancho}, p_{prof}, libre_{alto * ancho * prof} {}
    bool insertar_objeto(double volumen) {
        if (volumen <= 0 || volumen > libre_) return false;
        libre_ -= volumen; llena_ = libre_ <= 0; return true;
    }
    void vaciar() { libre_ = a_ * w_ * p_; llena_ = false; }
    double libre() const { return libre_; }
    bool llena() const { return llena_; }
    friend ostream& operator<<(ostream& os, const Box& b) { return os << "Caja " << b.a_ << 'x' << b.w_ << 'x' << b.p_
    ↪  << " libre=" << b.libre_ << (b.llena_ ? " (llena)" : ""); }
```

```
private:
    double a_, w_, p_, libre_;
    bool llena_ = false;
};
```

3 Entrada robusta

Si `in >> v` falla, el flujo queda en estado de error y **todas** las lecturas siguientes fallan. Hay que `clear()` y descartar la línea con `ignore`. Se lee en un bucle hasta recibir un número válido y positivo.

CPP

```
double leer_positivo(istream& in, const string& pregunta) {       // robusto frente a «abc»
    double v;
    while (true) {
        cout << pregunta;
        if (in >> v && v > 0) return v;
        in.clear();                                                 // limpia el estado de error
        in.ignore(numeric_limits<streamsize>::max(), '\n');         // descarta la línea mala
        cout << "valor no valido\n";
    }
}
```

4 main

CPP

```
int main() {
    Sphere s{2, Point_3D{0, 0, 0}};
    cout << s << '\n';
    cout << boolalpha << "(0.5,0,0) dentro: " << s.contiene({0.5, 0, 0}) << ", (1,0,0) en superficie: " <<
    ↪  s.en_superficie({1, 0, 0}) << '\n';
    Box b{1, 1, 1};
    cout << b.insertar_objeto(0.6) << ' ' << b.insertar_objeto(0.6) << ' ' << b.insertar_objeto(0.4) << '\n' << b <<
    ↪  '\n';
    b.vaciar(); cout << b << '\n';
    istringstream simulado{"abc\n-3\n4.5\n"};                        // en el examen sería cin
    cout << "diametro leido: " << leer_positivo(simulado, "diametro? ") << '\n';
}
```

Resultado final

Salida

```
Esfera d=2 centro (0, 0, 0) area=12.5664 volumen=4.18879
(0.5,0,0) dentro: true, (1,0,0) en superficie: true
true false true
Caja 1x1x1 libre=0 (llena)
Caja 1x1x1 libre=1
diametro leido: diametro? valor no valido
diametro? valor no valido
diametro? 4.5
```

Comprobación: con $d = 2$ ($r = 1$), área $4\pi = 12{,}566$ y volumen $\frac{4}{3}\pi = 4{,}189$. La caja de volumen 1 admite 0,6 y 0,4 pero rechaza el segundo 0,6, y queda llena (libre = 0).